

DevOps HQ Consolidated Dashboard – UI Design Doc

Status: Design Proposal
Date: 2026-04-09
Author: Selym (Subagent)
Timeout Context: 20-minute scope delivery

1. UI Component Overview

Refresh Button Placement

Location: Top-right corner of DevOps HQ header (next to existing controls)
Position: Aligned with Settings/Help icons
Visual: Primary action button (high contrast)
- Label: “Refresh Report” or icon-only (↻) - **Icon:** Standard refresh/reload icon - **Disabled state:** While report is generating - **Loading state:** Spinner overlay on icon, button disabled

Visual Hierarchy

DevOps HQ	[Refresh ↻] [⚙] [?]
📄 Consolidated Report (Last updated: 2m ago)	
TASK STATUS BREAKDOWN	
Proposed: 3 In-Progress: 5 Completed: 12 (47%)	
Blocked: 2 ⚠	
CRITICAL BLOCKERS (Need Attention)	
• [Task #104] Missing API schema from Backend team Owner: Ward Risk: 🟡 HIGH Action: Unblock	
• [Task #087] Design approval pending Owner: Jamie Risk: 🟡 MEDIUM Action: Ping	

TASK SNAPSHOT TABLE					
ID	Title	Status	Owner	Risk	
#104	API Schema Design	Blocked	Ward		
#103	DB Migration	In-Progress	Morgan		
#102	Auth Module	Completed	Selym		
#101	Cache Layer	In-Progress	Park		
...	...	...	...	...	...
Export Report: [PDF] [CSV] [Slack]					

2. Report Generation Trigger

Primary: onclick Handler

- **Button click** → Immediately send request to backend API endpoint
- **Real-time:** No caching; always fetch fresh data
- **Debounce:** Prevent double-clicks (500ms debounce)

Secondary: Auto-Refresh (Optional Phase 2)

- Configurable interval (default: 5 min)
- Manual toggle in settings: “Auto-refresh dashboard”
- Visual indicator when auto-refresh is active

Backend Integration (Coordinate with Ward)

Endpoint: POST /api/v1/dashboard/consolidated-report - Input: Optional filters (team, project, status) - Response: Aggregated task/blocker/risk snapshot - Timeout: 10s server-side

3. Report Display & Preview

Real-Time Preview Flow

User clicks "Refresh Report"

↓

UI shows loading overlay (toast/spinner)

↓

API calls POST /api/dashboard/consolidated-report

↓

Data streams back (or bulk returns in <10s)

↓
Preview renders in main dashboard area
↓
User can scroll, filter, or export
↓
(Optional) User clicks "Save Snapshot" to archive

Display Zones (Collapsible)

1. Summary Cards (Top)

- Total tasks, % complete, blocked count, at-risk count
- Color-coded risk indicators

2. Critical Blockers (Pinned)

- Only HIGH-risk blockers appear by default
- Sortable by owner, task ID, age
- Inline action buttons: "Unblock", "Escalate", "Reassign"

3. Task Snapshot Table

- Sortable columns: ID, Title, Status, Owner, Risk Level, Updated At
- Filterable by status, owner, risk level
- Expandable rows (show subtasks, comments, related blockers)

4. Export Options

- PDF: Styled report (good for meetings/archive)
 - CSV: Raw data for further analysis
 - Slack: Post summary to #devops-updates
-

4. Visible Fields in Consolidated View

Per Task:

- Task ID (link to full task details)
- Title / Description snippet
- Current Status (proposed | in-progress | completed | blocked)
- Assigned Owner (avatar + name)
- Risk Level (□ low | □ medium | □ high)
- Last Updated timestamp
- % Progress (for in-progress tasks)

Per Blocker:

- Blocker ID / Task it's blocking
- Description ("waiting for X", "missing Y approval")
- Assigned Owner (who's responsible for resolving)
- Risk Level (severity of impact)
- Age (how long it's been blocking)
- Suggested Action (unblock, escalate, reassign)

Summary Metrics:

- Total tasks: breakdown by status
- Velocity: completed/week (trend)
- Blocked count + % of total
- At-risk count (HIGH or MEDIUM risk)
- Owner workload (tasks per person)

5. Rough Mockup / Wireframe

Wire Layout (ASCII)

DevOps HQ Dashboard

Refresh

Status Overview

12 Completed (47%)

5 In-Progress

3 Proposed

2 Blocked

3 At-Risk (HIGH/MEDIUM)

At Risk Critical Blockers (2)

#104 API Schema Design

Age: 3 days

Status: Blocked Owner: Ward Risk: HIGH

Reason: Backend team hasn't delivered schema

Action: [Unblock] [Escalate] [Reassign]

#087 Design Approval

Age: 1 day

Status: Blocked Owner: Jamie Risk: MEDIUM

Reason: Pending stakeholder sign-off

Action: [Unblock] [Escalate] [Reassign]

Task Snapshot

ID	Title	Status	Owner	Risk
#104	API Schema Design	Blocked	Ward	
#103	DB Migration	In-Prog	Morgan	
#102	Auth Module	Completed	Selym	
#101	Cache Layer	In-Prog	Park	
#100	Frontend Redesign	Proposed	Jamie	
[Load more... 20 of 25]				

Export: [📄 PDF] [📄 CSV] [📄 Slack] | Last updated: 2 min ago

6. Architecture Recommendation

Recommended: Hybrid API + CLI Fallback

Primary Path: REST API Endpoint

```
POST /api/v1/dashboard/consolidated-report
├ Input: { filters: { team?, status?, owner? } }
├ Output: {
│   summary: { total, completed, inProgress, blocked, atRisk },
│   blockers: [ { id, taskId, reason, owner, risk, age } ],
│   tasks: [ { id, title, status, owner, risk, updated } ],
│   generatedAt: ISO8601
│ }
├ Latency target: <5s (cold), <1s (cached)
└ Cache: 3-5 min TTL
```

Why API-first: - Real-time updates in UI - Easy filtering/sorting client-side - Dashboard can live-refresh without page reload - Scalable (can cache at CDN level)

Secondary Path: CLI Script (Ward's backend)

```
devops-cli report:consolidated \
  --format json \
  --output /tmp/report.json
```

Use case: Scheduled nightly reports, archival, Slack automation

Integration Points:

- API calls CLI under the hood (if no real-time DB queries available)
 - CLI generates static JSON; API serves it
 - Webhook triggers: deploy completion → auto-refresh dashboard
-

7. Coordination with Ward (Backend)

Handoff Points for Backend Dev:

1. **Data Model** (Ward to define):
 - How are “blockers” stored? (separate table? field on task?)
 - Risk level calculation: automatic or manual?
 - Owner assignment: from task_assignee or separate?
2. **API Endpoint Requirements:**
 - Endpoint: POST /api/v1/dashboard/consolidated-report

- Response format (see schema above)
 - Filtering support: status, owner, risk
 - Pagination for tasks (limit/offset)
 - 3. Performance:**
 - How to keep query <5s for 100+ tasks?
 - Caching strategy? Redis? DB view?
 - Real-time updates: polling vs. WebSocket?
 - 4. Data Sources:**
 - Task table structure (assuming Supabase)
 - Blocker resolution logic
 - Risk scoring (automatic from deps/dates or manual field?)
 - 5. Testing:**
 - Mock API responses for frontend dev
 - Load test: 1000 tasks + concurrent requests
 - Edge cases: no blockers, all tasks blocked, missing data
-

8. Implementation Plan

Phase 1: MVP (1 week)

- ☐ Design React component: Refresh button + layout
- ☐ Mock API response (static JSON)
- ☐ Render task summary cards + blockers list
- ☐ Export to CSV
- ☐ Wire up onclick → mock API call

Phase 2: Backend Integration (1 week)

- ☐ Ward builds API endpoint
- ☐ Connect component to real endpoint
- ☐ Add filtering/sorting
- ☐ Implement caching strategy

Phase 3: Polish (3-5 days)



- ☐ PDF export (use jsPDF or react-pdf)
 - ☐ Slack integration (webhook post)
 - ☐ Auto-refresh toggle + scheduling
 - ☐ Real-time updates (WebSocket or polling)
 - ☐ Mobile-responsive design
-

9. Time Estimates

Frontend (UI Component)

- Refresh button + layout: **2-3 hours**
- Task snapshot table + sorting: **4-5 hours**
- Blockers display + actions: **3-4 hours**
- Export (CSV/PDF): **4-6 hours**
- Polish & testing: **2-3 hours**
- **Subtotal: ~16-21 hours (2 days)**

Backend (API Endpoint)

- Data aggregation logic: **4-6 hours**
- Blocker detection/ranking: **3-4 hours**
- Risk scoring: **2-3 hours**
- Caching layer: **2-3 hours**
- Testing & perf tuning: **3-4 hours**
- **Subtotal: ~14-20 hours (2 days)** ← Ward owns

Integration & Testing

- End-to-end tests: **2-3 hours**
- Load testing: **2-3 hours**
- **Subtotal: ~4-6 hours (0.5 days)**

Total: ~4-5 days (2 devs in parallel)

10. Tech Stack Recommendation

- **Frontend:** React + TypeScript
 - Component: ConsolidatedReportDashboard
 - State: React Query (for API caching + refetch)
 - Table: TanStack Table (React Table) for sorting/filtering

- Export: csv-download + jsPDF libraries
 - **Backend:** (Ward's choice, likely Supabase + Node.js)
 - Endpoint: Express/Hono middleware
 - Query: Supabase RLS + SQL aggregation
 - Cache: Redis (optional, for <5s latency)
 - **Styling:** Tailwind CSS + component library (match existing HQ style)
-

11. Success Criteria

- ☐ Refresh button loads real data in <5 seconds
 - ☐ Report shows all required fields (status, blockers, owner, risk)
 - ☐ Blockers are highlighted and actionable
 - ☐ Export works (CSV + PDF)
 - ☐ Mobile-responsive
 - ☐ Sorting/filtering functional
 - ☐ No duplicate reports (debounce working)
 - ☐ Error handling (API down, no data, etc.)
-

Next Steps

1. **Ward:** Review backend data model & API contract
2. **Frontend:** Start component skeleton + mock data
3. **Design:** Finalize styling (match existing HQ look)
4. **Dev:** Parallel frontend + backend work
5. **Integration:** Connect real API end of week 1